

Chapter 49 — Bit Manipulation

0. What You Will Learn in This Chapter

Bit Manipulation is the art of solving problems using the binary representation of numbers.

It is heavily used in:

- XOR problems
- subset enumeration
- masks
- parity checks
- power-of-two checks
- counting set bits
- finding unique numbers
- optimization tricks
- bitmask DP
- low-level integer logic
- binary tries
- mathematical interview problems

By the end of this chapter, you should understand:

- Binary representation
- AND, OR, XOR, NOT
- Left shift and right shift
- Check, set, clear, toggle bits
- Count set bits
- Lowbit
- Power of two checks
- XOR cancellation
- Single Number variants
- Missing number with XOR
- Range bitwise AND
- Subset generation using masks
- Bitmask enumeration
- Gray code
- Maximum XOR logic
- Common bit tricks and traps

Core problems covered in this chapter:

1. Number of 1 Bits
2. Counting Bits
3. Power of Two
4. Power of Four
5. Single Number
6. Single Number II
7. Single Number III
8. Missing Number
9. Reverse Bits
10. Hamming Distance
11. Total Hamming Distance
12. Sum of Two Integers

- 13. Bitwise AND of Numbers Range
 - 14. Subsets using bitmask
 - 15. Maximum XOR of Two Numbers, binary trie review
 - 16. Gray Code
 - 17. Minimum Flips to Make a OR b Equal c
 - 18. Find the Difference
 - 19. UTF-8 Validation, preview
 - 20. Bitmask DP connection
-

1. What Is Bit Manipulation?

Every integer is stored in binary.

Example:

13

in binary is:

1101

Meaning:

$8 + 4 + 1 = 13$

Bits are positions containing 0 or 1.

Bit manipulation uses operations directly on these bits.

2. Why Bit Manipulation Matters

Bit tricks often turn complicated logic into simple operations.

Examples:

x is even if lowest bit is 0
x is power of two if x has exactly one set bit
a XOR a = 0
a XOR 0 = a

Bit manipulation is also important for:

- compact state representation
 - subset enumeration
 - performance
 - cryptic interview tricks
 - low-level systems knowledge
-

3. Binary Basics

3.1 Powers of Two

Binary places:

bit 0 -> 1
bit 1 -> 2
bit 2 -> 4

```
bit 3 -> 8  
bit 4 -> 16
```

Example:

```
binary 10110
```

means:

```
16 + 4 + 2 = 22
```

3.2 Set Bit

A bit is set if it is 1.

Example:

```
22 = 10110
```

Set bits are at positions:

```
1, 2, 4
```

4. Core Bit Operators

4.1 AND &

Bit is 1 only if both bits are 1.

```
1100  
1010  
----  
1000
```

Use for checking bits and masking.

4.2 OR |

Bit is 1 if either bit is 1.

```
1100  
1010  
----  
1110
```

Use for setting bits.

4.3 XOR ^

Bit is 1 if bits are different.

```
1100  
1010  
----  
0110
```

Use for toggling and cancellation.

4.4 NOT ~

Flips bits.

Be careful because integers have fixed width and signed representation.

4.5 Left Shift <<

```
1 << i
```

means:

```
2^i
```

Example:

```
1 << 3 = 8
```

4.6 Right Shift >>

Shifts bits right.

```
x >> 1
```

roughly divides non-negative x by 2.

5. Essential Bit Operations

5.1 Check Bit i

```
bool isSet = (x & (1 << i)) != 0;
```

5.2 Set Bit i

```
x = x | (1 << i);
```

5.3 Clear Bit i

```
x = x & ~(1 << i);
```

5.4 Toggle Bit i

```
x = x ^ (1 << i);
```

5.5 Count Set Bits

```
__builtin_popcount(x)
```

For long long:

```
__builtin_popcountll(x)
```

5.6 Lowest Set Bit

```
x & -x
```

Example:

```
x = 12 = 1100
-x two's complement
x & -x = 0100 = 4
```

5.7 Remove Lowest Set Bit

```
x = x & (x - 1);
```

6. XOR Properties

XOR is extremely important.

6.1 Same Number Cancels

$$x \oplus x = 0$$

6.2 Zero Identity

$$x \oplus 0 = x$$

6.3 Order Does Not Matter

$$a \oplus b \oplus a = b$$

Because:

$$a \oplus a = 0$$

$$0 \oplus b = b$$

6.4 Used In

- Single Number
 - Missing Number
 - Find Difference
 - XOR ranges
 - bitmask parity
-

7. When to Use Bit Manipulation

Use bit manipulation when problem mentions:

7.1 Binary Bits

Examples:

- count bits
- reverse bits
- hamming distance

7.2 XOR / Unique Number

Examples:

- every number appears twice except one
- find missing number
- find two unique numbers

7.3 Power of Two

Use bit trick.

7.4 Subsets / Masks

Use bitmask enumeration.

7.5 Bitwise AND/OR Range

Use bit-prefix logic.

7.6 Constraints Are Small n

For subsets:

$$n \leq 20$$

often suggests masks.

8. When Not to Use Bit Manipulation

Do not force bit manipulation when:

8.1 A Simple Arithmetic Solution Is Clearer

Sometimes modulo or division is easier.

8.2 Values Are Floating Point

Bit operations are for integers.

8.3 You Need Ordered Data

Use sorting, heap, tree, etc.

8.4 You Need Large Arbitrary Precision

Use strings or big integers.

9. Pattern Recognition Checklist

Ask:

21. Does the problem mention binary representation?
 22. Are there repeated numbers and one unique?
 23. Does XOR cancellation apply?
 24. Do I need count of set bits?
 25. Is this a power of two/four?
 26. Do I need generate subsets?
 27. Is a mask useful?
 28. Do I need check/set/clear/toggle bits?
 29. Is range bitwise AND involved?
 30. Are there only 32 or 64 bit positions to inspect?
-

10. Core Template 1 — Count Bits Manually

```
int countBits(int x) {
    int count = 0;

    while (x != 0) {
        count += x & 1;
        x >>= 1;
    }

    return count;
}
```

Better:

```
while (x != 0) {
    x &= (x - 1);
    count++;
}
```

This loops only over set bits.

11. Core Template 2 — Iterate Bit Positions

```
for (int bit = 0; bit < 32; bit++) {  
    if (x & (1 << bit)) {  
        // bit is set  
    }  
}
```

For long long:

```
for (int bit = 0; bit < 64; bit++) {  
    if (x & (1LL << bit)) {  
    }  
}
```

12. Core Template 3 — Generate Subsets With Mask

```
vector<vector<int>> subsets(vector<int>& nums) {  
    int n = nums.size();  
  
    vector<vector<int>> result;  
  
    for (int mask = 0; mask < (1 << n); mask++) {  
        vector<int> subset;  
  
        for (int i = 0; i < n; i++) {  
            if (mask & (1 << i)) {  
                subset.push_back(nums[i]);  
            }  
        }  
  
        result.push_back(subset);  
    }  
  
    return result;  
}
```

13. Core Template 4 — Iterate Submasks

```
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {  
    // sub is a non-empty submask of mask  
}
```

This is important in bitmask DP.

14. Full Problem Walkthrough 1 — LeetCode 191: Number of 1 Bits

14.1 Problem Summary

Return number of set bits in unsigned integer n.

14.2 Basic Approach

Use:

```
n & (n - 1)
```

to remove lowest set bit.

14.3 C++ Code

```
class Solution {  
public:  
    int hammingWeight(uint32_t n) {  
        int count = 0;  
  
        while (n != 0) {  
            n &= (n - 1);  
            count++;  
        }  
  
        return count;  
    }  
};
```

14.4 Complexity

Time:

```
O(number of set bits)
```

Space:

```
O(1)
```

15. Full Problem Walkthrough 2 — LeetCode 338: Counting Bits

15.1 Problem Summary

Given n, return array ans where:

```
ans[i] = number of 1 bits in i
```

for:

```
0 <= i <= n
```

15.2 DP Bit Recurrence

Use:

```
bits[i] = bits[i >> 1] + (i & 1)
```

Because shifting right removes last bit.

15.3 C++ Code

```
class Solution {
public:
    vector<int> countBits(int n) {
        vector<int> bits(n + 1, 0);

        for (int i = 1; i <= n; i++) {
            bits[i] = bits[i >> 1] + (i & 1);
        }

        return bits;
    }
};
```

15.4 Alternative Recurrence

```
bits[i] = bits[i & (i - 1)] + 1;
```

This removes lowest set bit.

16. Full Problem Walkthrough 3 — LeetCode 231: Power of Two

16.1 Problem Summary

Return true if n is a power of two.

16.2 Key Insight

Power of two has exactly one set bit.

Examples:

```
1  = 0001
2  = 0010
4  = 0100
8  = 1000
```

For such n:

```
n & (n - 1) == 0
```

16.3 C++ Code

```
class Solution {
public:
    bool isPowerOfTwo(int n) {
        return n > 0 && (n & (n - 1)) == 0;
    }
};
```

17. Full Problem Walkthrough 4 — LeetCode 342: Power of Four

17.1 Problem Summary

Return true if n is a power of four.

Powers of four:

1, 4, 16, 64, ...

Binary:

```
1      -> bit 0
100    -> bit 2
10000  -> bit 4
```

The single set bit must be in an even position.

17.2 C++ Code

```
class Solution {
public:
    bool isPowerOfFour(int n) {
        if (n <= 0) {
            return false;
        }

        if ((n & (n - 1)) != 0) {
            return false;
        }

        return (n & 0x55555555) != 0;
    }
};
```

17.3 Why 0x55555555?

In binary, it has 1s at even positions:

010101010101010101010101010101

18. Full Problem Walkthrough 5 — LeetCode 136: Single Number

18.1 Problem Summary

Every number appears twice except one.

Find the unique number.

18.2 XOR Insight

Pairs cancel:

$$a \oplus a = 0$$

All duplicates vanish.

18.3 C++ Code

```
class Solution {
public:
    int singleNumber(vector<int>& nums) {
        int result = 0;

        for (int x : nums) {
            result ^= x;
        }

        return result;
    }
};
```

18.4 Complexity

Time:

$O(n)$

Space:

$O(1)$

19. Full Problem Walkthrough 6 — LeetCode 137: Single Number II

19.1 Problem Summary

Every number appears three times except one number that appears once.

Find the unique number.

19.2 Bit Count Insight

For each bit position:

- Count how many numbers have that bit set.
- Since duplicates appear 3 times, their contribution is multiple of 3.
- Unique number contributes remainder.

19.3 C++ Code

```
class Solution {
public:
    int singleNumber(vector<int>& nums) {
```

```

    int result = 0;

    for (int bit = 0; bit < 32; bit++) {
        int count = 0;

        for (int x : nums) {
            if ((x >> bit) & 1) {
                count++;
            }
        }

        if (count % 3 != 0) {
            result |= (1 << bit);
        }
    }

    return result;
}
};

```

19.4 Negative Numbers

This works with signed ints in typical 32-bit representation.

Be careful around bit 31, but C++ int behavior is accepted in LeetCode-style contexts.

20. Full Problem Walkthrough 7 — LeetCode 260: Single Number III

20.1 Problem Summary

Every number appears twice except two numbers that appear once.

Find the two unique numbers.

20.2 XOR Insight

XOR all numbers:

```
xorAll = a ^ b
```

Since $a \neq b$, xorAll has at least one set bit.

That set bit means a and b differ there.

Use it to split nums into two groups.

20.3 C++ Code

```

class Solution {
public:
    vector<int> singleNumber(vector<int>& nums) {
        int xorAll = 0;

        for (int x : nums) {
            xorAll ^= x;
        }
    }
}

```

```

        int diffBit = xorAll & -xorAll;

        int a = 0;
        int b = 0;

        for (int x : nums) {
            if (x & diffBit) {
                a ^= x;
            } else {
                b ^= x;
            }
        }

        return {a, b};
    }
};

```

20.4 Why It Works

Duplicates go into the same group and cancel.

The two unique numbers go into different groups because they differ on diffBit.

21. Full Problem Walkthrough 8 — LeetCode 268: Missing Number

21.1 Problem Summary

Given nums containing n distinct numbers from 0 to n, find missing number.

21.2 XOR Approach

XOR all indices and values.

Duplicates cancel.

21.3 C++ Code

```

class Solution {
public:
    int missingNumber(vector<int>& nums) {
        int n = nums.size();

        int result = n;

        for (int i = 0; i < n; i++) {
            result ^= i;
            result ^= nums[i];
        }

        return result;
    }
};

```

21.4 Alternative

Sum formula also works.

XOR avoids overflow.

22. Full Problem Walkthrough 9 — LeetCode 190: Reverse Bits

22.1 Problem Summary

Reverse bits of a 32-bit unsigned integer.

22.2 C++ Code

```
class Solution {
public:
    uint32_t reverseBits(uint32_t n) {
        uint32_t result = 0;

        for (int i = 0; i < 32; i++) {
            result <<= 1;
            result |= (n & 1);
            n >>= 1;
        }

        return result;
    }
};
```

22.3 Explanation

Take lowest bit of n and append it to result.

Repeat 32 times.

23. Full Problem Walkthrough 10 — LeetCode 461: Hamming Distance

23.1 Problem Summary

Hamming distance between x and y is number of positions where bits differ.

23.2 XOR Insight

Bits differ exactly where XOR has 1.

23.3 C++ Code

```
class Solution {
public:
    int hammingDistance(int x, int y) {
        int z = x ^ y;

        return __builtin_popcount(z);
    }
};
```

```
}  
};
```

24. Full Problem Walkthrough 11 — LeetCode 477: Total Hamming Distance

24.1 Problem Summary

Given nums, return sum of Hamming distances between all pairs.

24.2 Bit Contribution

For each bit:

```
ones = count of numbers with bit set  
zeros = n - ones
```

Pairs differing at this bit:

```
ones * zeros
```

Sum over all bits.

24.3 C++ Code

```
class Solution {  
public:  
    int totalHammingDistance(vector<int>& nums) {  
        int n = nums.size();  
  
        int answer = 0;  
  
        for (int bit = 0; bit < 32; bit++) {  
            int ones = 0;  
  
            for (int x : nums) {  
                if ((x >> bit) & 1) {  
                    ones++;  
                }  
            }  
  
            int zeros = n - ones;  
  
            answer += ones * zeros;  
        }  
  
        return answer;  
    }  
};
```

24.4 Complexity

Time:

$O(32n)$

Space:

$O(1)$

25. Full Problem Walkthrough 12 — LeetCode 371: Sum of Two Integers

25.1 Problem Summary

Calculate sum of two integers without using + or -.

25.2 Bit Insight

Addition has two parts:

```
sum without carry = a ^ b  
carry = (a & b) << 1
```

Repeat until no carry.

25.3 C++ Code

```
class Solution {  
public:  
    int getSum(int a, int b) {  
        while (b != 0) {  
            unsigned carry = (unsigned)(a & b) << 1;  
  
            a = a ^ b;  
            b = carry;  
        }  
  
        return a;  
    }  
};
```

25.4 Why unsigned Carry?

Avoid undefined behavior on left-shifting negative signed values.

26. Full Problem Walkthrough 13 — LeetCode 201: Bitwise AND of Numbers Range

26.1 Problem Summary

Given left and right, return bitwise AND of every number in range [left, right].

26.2 Key Insight

Only common binary prefix remains.
As range expands, lower changing bits become 0.

26.3 C++ Code

```
class Solution {
public:
    int rangeBitwiseAnd(int left, int right) {
        int shifts = 0;

        while (left < right) {
            left >>= 1;
            right >>= 1;
            shifts++;
        }

        return left << shifts;
    }
};
```

26.4 Example

```
5 = 101
6 = 110
7 = 111
```

Common prefix is:

1__

Answer:

100 = 4

27. Full Problem Walkthrough 14 — LeetCode 78: Subsets Using Bitmask

27.1 Problem Summary

Return all subsets of nums.

27.2 Bitmask Idea

Each mask represents one subset.

27.3 C++ Code

```
class Solution {
public:
    vector<vector<int>> subsets(vector<int>& nums) {
        int n = nums.size();

        vector<vector<int>> result;

        for (int mask = 0; mask < (1 << n); mask++) {
```

```

        vector<int> subset;

        for (int i = 0; i < n; i++) {
            if (mask & (1 << i)) {
                subset.push_back(nums[i]);
            }
        }

        result.push_back(subset);
    }

    return result;
};

```

27.4 Complexity

Time:

$O(n * 2^n)$

Space:

$O(n * 2^n)$

for output.

28. Full Problem Walkthrough 15 — LeetCode 89: Gray Code

28.1 Problem Summary

Generate sequence of n-bit Gray code.

Adjacent numbers differ by exactly one bit.

28.2 Formula

The i-th Gray code is:

$i \oplus (i \gg 1)$

28.3 C++ Code

```

class Solution {
public:
    vector<int> grayCode(int n) {
        vector<int> result;

        int total = 1 << n;

        for (int i = 0; i < total; i++) {
            result.push_back(i ^ (i >> 1));
        }

        return result;
    }
};

```

```
}  
};
```

29. Full Problem Walkthrough 16 — LeetCode 1318: Minimum Flips to Make a OR b Equal c

29.1 Problem Summary

Given a, b, c.

Return minimum bit flips in a and b so:

```
(a | b) == c
```

29.2 Bit-by-Bit Logic

For each bit:

If c bit is 1:

at least one of a or b must be 1

If both are 0, need 1 flip.

If c bit is 0:

both a and b must be 0

Need flip every 1 among a and b.

29.3 C++ Code

```
class Solution {  
public:  
    int minFlips(int a, int b, int c) {  
        int flips = 0;  
  
        for (int bit = 0; bit < 32; bit++) {  
            int abit = (a >> bit) & 1;  
            int bbit = (b >> bit) & 1;  
            int cbit = (c >> bit) & 1;  
  
            if (cbit == 1) {  
                if (abit == 0 && bbit == 0) {  
                    flips++;  
                }  
            } else {  
                flips += abit + bbit;  
            }  
        }  
  
        return flips;  
    }  
};
```

30. Full Problem Walkthrough 17 — LeetCode 389: Find the Difference

30.1 Problem Summary

String *t* is generated by shuffling *s* and adding one extra character.
Find the extra character.

30.2 XOR Approach

XOR all characters in both strings.
Common chars cancel.

30.3 C++ Code

```
class Solution {
public:
    char findTheDifference(string s, string t) {
        char result = 0;

        for (char c : s) {
            result ^= c;
        }

        for (char c : t) {
            result ^= c;
        }

        return result;
    }
};
```

31. Full Problem Walkthrough 18 — Maximum XOR Review

31.1 Problem Summary

Find max XOR of two numbers.

31.2 Best Techniques

Two approaches:

31. Binary Trie

32. Greedy bit-by-bit with prefix set

31.3 Greedy Prefix Set Code

```
class Solution {
public:
    int findMaximumXOR(vector<int>& nums) {
        int answer = 0;
```

```

int mask = 0;

for (int bit = 31; bit >= 0; bit--) {
    mask |= (1 << bit);

    unordered_set<int> prefixes;

    for (int x : nums) {
        prefixes.insert(x & mask);
    }

    int candidate = answer | (1 << bit);

    for (int p : prefixes) {
        if (prefixes.count(p ^ candidate)) {
            answer = candidate;
            break;
        }
    }

    return answer;
}
};

```

31.4 Why It Works

Build answer from high bit to low bit.

At each bit, greedily test whether this bit can be 1.

32. Common Bit Edge Cases

32.1 Negative Numbers

Signed shifts can be tricky.

Use unsigned when needed.

32.2 Overflow of $1 \ll 31$

Use:

```
1LL << bit
```

or unsigned.

32.3 $n = 0$

Power of two should return false for 0.

32.4 Duplicate Numbers

XOR cancellation depends on exact duplicate counts.

32.5 Bit Width

Know whether problem uses:

32-bit

64-bit

33. Common Bit Mistakes

Mistake 1 — Missing Parentheses

Wrong:

```
x & 1 << i
```

Better:

```
x & (1 << i)
```

Mistake 2 — Power of Two Without $n > 0$

```
(n & (n - 1)) == 0
```

is true for $n = 0$, so check $n > 0$.

Mistake 3 — Signed Shift Issues

Use unsigned for low-level shift problems.

Mistake 4 — Confusing OR and XOR

OR sets bits.

XOR toggles/cancels bits.

Mistake 5 — Assuming XOR Works for Triplicates

Single Number II needs bit counts modulo 3, not simple XOR.

34. Interview Explanation Templates

34.1 Single Number

XOR cancels equal pairs because $x \text{ XOR } x$ is 0. After XORing all numbers, only the unique number remains.

34.2 Power of Two

A power of two has exactly one set bit. Subtracting one flips all lower bits, so $n \& (n - 1)$ becomes 0.

34.3 Range Bitwise AND

The AND of a range keeps only the common binary prefix of left and right. All changing lower bits become 0.

34.4 Single Number III

XOR of all numbers gives a XOR b. I use a set bit from that value to separate numbers into two groups, where duplicates cancel inside each group.

34.5 Total Hamming Distance

For each bit, every pair with different bit contributes 1. If ones is count of set bits and zeros is $n - \text{ones}$, contribution is $\text{ones} \times \text{zeros}$.

35. Complexity Patterns

Problem Type	Time	Space
Count bits of one number	$O(32)$ or $O(\text{set bits})$	$O(1)$

Problem Type	Time	Space
Counting Bits 0..n	$O(n)$	$O(n)$
Power of Two	$O(1)$	$O(1)$
Single Number	$O(n)$	$O(1)$
Single Number II	$O(32n)$	$O(1)$
Single Number III	$O(n)$	$O(1)$
Hamming Distance	$O(1)$	$O(1)$
Total Hamming Distance	$O(32n)$	$O(1)$
Range Bitwise AND	$O(32)$	$O(1)$
Subsets by mask	$O(n2^n)$	output
Maximum XOR Trie	$O(32n)$	$O(32n)$

36. Expanded Practice Set

36.1 Must-Do Easy / Medium

- 33. Number of 1 Bits
- 34. Counting Bits
- 35. Power of Two
- 36. Power of Four
- 37. Single Number
- 38. Missing Number
- 39. Reverse Bits
- 40. Hamming Distance
- 41. Find the Difference
- 42. Subsets

36.2 Strong Interview Practice

- 43. Single Number II
- 44. Single Number III
- 45. Total Hamming Distance
- 46. Sum of Two Integers
- 47. Bitwise AND of Numbers Range
- 48. Minimum Flips to Make a OR b Equal c
- 49. Gray Code
- 50. Maximum XOR of Two Numbers
- 51. UTF-8 Validation

36.3 Hard / Stretch

- 52. Maximum XOR With an Element From Array
- 53. Bitwise ORs of Subarrays
- 54. Minimum XOR Sum of Two Arrays, bitmask DP

- 55. Smallest Sufficient Team, bitmask DP
 - 56. Count Triplets That Can Form Two Arrays of Equal XOR
 - 57. Maximum Product of Word Lengths
 - 58. Find K-th Bit in N-th Binary String
-

37. Suggested Study Order

Step 1 — Basic Operations

Solve:

- 59. Number of 1 Bits
- 60. Counting Bits
- 61. Power of Two

Goal:

Understand set bits and $n \& (n - 1)$.

Step 2 — XOR Tricks

Solve:

- 62. Single Number
- 63. Missing Number
- 64. Find the Difference

Goal:

Master cancellation.

Step 3 — Advanced XOR

Solve:

- 65. Single Number II
- 66. Single Number III
- 67. Maximum XOR

Goal:

Learn bit counts, partitioning, and tries.

Step 4 — Bit Position Contribution

Solve:

- 68. Hamming Distance
- 69. Total Hamming Distance
- 70. Minimum Flips

Goal:

Think bit-by-bit.

Step 5 — Masks

Solve:

- 71. Subsets
- 72. Gray Code
- 73. Bitmask DP review problems

Goal:

Use masks as sets.

38. Pattern Decision Table

Problem Clue	Bit Technique
count 1 bits	popcount / n & n-1
power of two	n & (n-1)
every number twice except one	XOR all
every number thrice except one	bit counts mod 3
two unique numbers	xorAll + lowbit partition
missing from 0..n	XOR indices and values
hamming distance	popcount(x ^ y)
total hamming distance	per-bit contribution
range AND	common prefix
subsets	bitmask enumeration
maximum XOR	binary trie / greedy prefixes
flips to satisfy OR	bit-by-bit logic
add without plus	XOR + carry

39. Mini Problem 1 — Check If Bit Alternates

Given n, check if binary bits alternate.

Example:

```
1010 -> true
111 -> false
bool hasAlternatingBits(int n) {
    int x = n ^ (n >> 1);

    return (x & (x + 1)) == 0;
}
```

Why?

If bits alternate, x becomes all 1s.

40. Mini Problem 2 — Maximum Product of Word Lengths

Given words, find max product of lengths of two words with no common letters.

Use bitmask for each word.

```
class Solution {
public:
    int maxProduct(vector<string>& words) {
        int n = words.size();

        vector<int> mask(n, 0);

        for (int i = 0; i < n; i++) {
            for (char c : words[i]) {
                mask[i] |= 1 << (c - 'a');
            }
        }

        int answer = 0;

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if ((mask[i] & mask[j]) == 0) {
                    int product = words[i].size() * words[j].size();
                    answer = max(answer, product);
                }
            }
        }

        return answer;
    }
};
```

41. Mini Problem 3 — Count Triplets With Equal XOR

Problem idea:

If:

$\text{xor}(i..j-1) == \text{xor}(j..k)$

then:

$\text{xor}(i..k) == 0$

Can be optimized with prefix XOR.

This is a useful XOR + prefix pattern.

42. Mini Problem 4 — Bitwise ORs of Subarrays Preview

For each position, maintain all possible OR values of subarrays ending there.
Number of distinct OR values per position is small because bits only turn on.
This is an advanced bitset-like set DP pattern.

43. Common Edge Case Checklist

Before submitting bit code, test:

```
0
1
2
3
4
negative numbers
INT_MIN
INT_MAX
all duplicates
single element
two unique values
range left == right
range crosses power of two
n = 31 for shifts
large arrays
```

44. Debugging Bit Manipulation

If bit code fails, check:

- 74. Are you using parentheses around shifts?
 - 75. Are you handling $n = 0$?
 - 76. Are signed shifts safe?
 - 77. Should you use unsigned or long long?
 - 78. Are you checking all 32 bits?
 - 79. Is XOR cancellation valid for the frequency pattern?
 - 80. Is bit 31 handled correctly?
 - 81. Are you using OR when you need XOR?
 - 82. Are masks initialized correctly?
-

45. What to Master Before Chapter 50

Before moving to Math / Number Theory, make sure you can:

- Explain AND, OR, XOR, shifts
- Check/set/clear/toggle bits
- Count set bits
- Use $n \& (n - 1)$
- Use $x \& -x$
- Solve Single Number I, II, III
- Solve Missing Number
- Solve Counting Bits

- Solve Range Bitwise AND
 - Solve Total Hamming Distance
 - Generate subsets with masks
 - Understand bitmask DP connection
 - Understand binary Trie XOR connection
-

46. Final Cheat Sheet

Check Bit

```
x & (1 << i)
```

Set Bit

```
x | (1 << i)
```

Clear Bit

```
x & ~(1 << i)
```

Toggle Bit

```
x ^ (1 << i)
```

Power of Two

```
n > 0 && (n & (n - 1)) == 0
```

Remove Lowest Set Bit

```
x &= (x - 1)
```

Lowest Set Bit

```
x & -x
```

Count Bits

```
__builtin_popcount(x)
```

XOR Cancellation

```
a ^ a = 0  
a ^ 0 = a
```

Single Number

```
result ^= x
```

Range AND

```
while (left < right) {  
    left >>= 1;  
    right >>= 1;  
    shifts++;  
}
```

```
return left << shifts;
```

Subsets

```
for (int mask = 0; mask < (1 << n); mask++)
```

47. Core Takeaway

Bit Manipulation is about using binary representation directly.

The most important ideas are:

- masks represent sets
- XOR cancels equal values
- $n \& (n - 1)$ removes the lowest set bit
- powers of two have one set bit
- bitwise range AND keeps common prefix
- many problems can be solved by checking all 32 bit positions

Bit tricks look magical at first, but most come from a few simple properties.

Master:

```
AND for checking  
OR for setting  
XOR for toggling/cancellation  
SHIFT for powers of two
```

and the rest becomes much easier.